

CLI-инструменты для AI-агентов с открытым исходным кодом — аналитический отчёт и сравнение функциональности

v1.0 @ 02.2026
Written by: Andrei Leman & Claude Opus 4.6

Дата создания: 26.02.2026 | Источники: 42 элемента базы знаний из 4 просканированных репозиториях + веб-исследование
Источники базы знаний: OpenCode (262), Crush (263), awesome-opencode (264), Codex (265)

1. Краткое резюме

Рынок CLI-инструментов для AI-агентов кодирования к началу 2026 года сформировался в зрелую конкурентную экосистему, насчитывающую около 15 заметных инструментов. Ключевые тенденции:

- Go и Rust доминируют** в новых реализациях CLI (OpenCode, Crush, Codex-rs, Goose), вытесняя Python/TypeScript за счёт производительности
- MCP (Model Context Protocol)** стал универсальным стандартом расширяемости — каждый крупный инструмент поддерживает его или планирует поддержку
- Мультипровайдерная поддержка** является обязательным минимумом — инструменты поддерживают 7–12+ провайдеров LLM через уровни абстракции
- Интеграция с LSP** отделяет серьёзные инструменты от игрушек — только OpenCode, Crush и Zed имеют глубокую поддержку LSP
- Песочница (sandboxing)** — новый рубеж — Codex лидирует с Seatbelt/Landlock, остальные полагаются на запросы разрешений
- Паттерн «агент как сервер»** набирает популярность — Codex может выступать в роли MCP-сервера, обеспечивая мета-агентные архитектуры
- TUI-фреймворки сходятся** к Bubble Tea (Go) и Ratatui (Rust) для создания богатых терминальных интерфейсов

Позиционирование на рынке (февраль 2026)

Уровень	Инструменты	Звёзды
Доминирующие	Gemini CLI (~95K), Zed (~76K), Cline (~58K), Codex (~49K), Claude Code (~41K), Aider (~41K)	50K+
Сильные	Continue (~32K), Goose (~31K), OpenCode (~30K), Warp (~26K)	25–50K
Растущие	Qwen Code (~15K), Plandex (~15K), Crush (~14K), Codebuff (~3K)	<15K

2. Детальный обзор инструментов

2.1 OpenCode (opencode-ai/opencode)

Обзор: Терминальный AI-ассистент для кодирования на Go с TUI на Bubble Tea. ~29,7K звёзд, Apache-2.0.

Архитектура:

- `cmd/` — точка входа CLI
- `internal/app` — оркестрация сервисов
- `internal/llm` — цикл агента + провайдеры + инструменты + промпты
- `internal/tui` — интерфейс на Bubble Tea
- `internal/lsp` — интеграция с Language Server Protocol
- `internal/db` — хранение в SQLite через sqlc
- Конфигурация: на основе Viper, `~/.opencode.json` или `./.opencode.json`

Цикл агента: Однопоточный цикл ReAct. `processGeneration()` входит в цикл использования инструментов, выполняет инструменты последовательно с проверкой разрешений. Цикл продолжается, пока причина завершения — `toolUse`. Отмена через `sync.Map` с ключом по `sessionId`.

Инструменты (12+):

Инструмент	Описание
bash	Постоянная сессия оболочки, тайм-аут по умолчанию 60 с / максимум 600 с. Безопасные команды не требуют подтверждения. Запрещены: curl, wget, nc, telnet, браузеры
edit	Замена текста по диапазону строк с обнаружением конфликтов
write	Создание/обновление файлов, автоматическое создание директорий, проверка времени модификации относительно последнего чтения
view	Чтение файлов с указанием диапазона строк
patch	Применение унифицированных diff-патчей с обнаружением нечёткого совпадения (отклоняется при fuzz > 3), атомарное применение
glob	Поиск файлов по шаблону
grep	Двойная стратегия: сначала ripgrep, затем fallback на regex. Лимит 100 файлов, максимум 200 совпадений
ls	Листинг директорий
fetch	HTTP-запросы
sourcegraph	Поиск кода по репозиториям через Sourcegraph API
agent	Порождение подагентов (подмножество инструментов только для чтения: glob, grep, ls, sourcegraph, view)
diagnostics	Диагностика LSP (при наличии LSP)

Поддержка провайдеров: 7+ провайдеров — OpenAI, Anthropic, Google, Azure, AWS Bedrock, Groq, OpenRouter, а также любые API, совместимые с OpenAI.

MCP: Клиентская поддержка MCP для внешних серверов инструментов.

Интеграция с LSP: Глубокая интеграция через `internal/lsp`. Инструмент диагностики запрашивает языковые серверы. Инструменты `write` и `patch` ожидают диагностику LSP после изменения файлов для обнаружения синтаксических ошибок.

TUI: Фреймворк Bubble Tea со стилизацией Lip Gloss. Многопанельная компоновка.

Управление сессиями: На основе SQLite через `sqlc`. Сохранение сессий, история сообщений, автоматическая генерация заголовков.

Система разрешений: Пакет `internal/permission`. Безопасные команды только для чтения (`git status`, `go test`, `ls`) не требуют подтверждения. Файловые операции требуют предварительного чтения. Отказ в разрешении останавливает оставшиеся выполнения инструментов.

Уникальные возможности:

- Интеграция с Sourcegraph для поиска по нескольким репозиториям
- Подагент с ограниченным набором инструментов только для чтения
- Диагностика LSP интегрирована в рабочие процессы `write/patch`
- Многоуровневая конфигурация на основе Viper (пользовательская + проектная)

2.2 Crush (charmbracelet/crush)

Обзор: Терминальный AI-агент для кодирования от Charmbracelet. Go 1.26, ~14,4К звёзд. Построен на фреймворке Fantasy (`charm.land/fantasy`).

Архитектура:

Coordinator → SessionAgent → Tools

↓

OAuth/Auth

Multi-provider

Session mgmt

↓

Agent Loop

Summarize

Title gen

↓

20+ инструментов

Интеграция MCP

Интеграция LSP

- `coordinator.go` — оркестратор верхнего уровня, управляет сессиями, создаёт агентов
- `agent.go` — цикл агента для каждой сессии, очередь сообщений, автоматическое резюмирование
- `tools/*.go` — 20+ встроенных инструментов через интерфейс Fantasy AgentTool

- `tools/mcp/` — полная поддержка MCP (транспорты `stdio/HTTP/SSE`)
- `hyper/provider.go` — собственный LLM-прокси `Charmbracelet` (`hyper.charm.land`)
- `prompt/` — система шаблонов `Go text/template`, встраивание на этапе компиляции

Инструменты (20+):

Инструмент	Описание
<code>bash</code>	Выполнение команд оболочки с поддержкой фоновых процессов. Автоматический перевод в фон через 60 с. Чёрный список: <code>curl</code> , <code>wget</code> , <code>ssh</code> , <code>sudo</code> , <code>apt</code> , <code>brew</code> и др.
<code>edit</code>	Редактирование одного файла с обязательным предварительным чтением, проверкой времени модификации
<code>multi_edit</code>	Пакетное последовательное редактирование одного файла
<code>view</code>	Чтение файлов, добавляет диагностику LSP при просмотре
<code>glob</code>	Поиск файлов по шаблону
<code>grep</code>	Поиск по содержимому
<code>ls</code>	Листинг директорий
<code>write</code>	Создание файлов
<code>fetch</code>	HTTP-запросы (лимит 5 МБ, тайм-аут 600 с, только <code>http/https</code>)
<code>download</code>	Скачивание файлов
<code>web_search</code>	Поиск через <code>DuckDuckGo</code> (API-ключ не требуется)
<code>sourcegraph</code>	Поиск по репозиториям через <code>Sourcegraph GraphQL API</code> (максимум 20 результатов)
<code>agent</code>	Порождение подагентов для делегированных задач
<code>agentic_fetch</code>	Выделенный подагент для веб-исследований со своим набором инструментов
<code>diagnostics</code>	Диагностика LSP (по файлу + по всему проекту, сортировка по серьёзности)
<code>references</code>	LSP <code>FindReferences</code> с предварительной фильтрацией через <code>grep</code> , поддержка квалифицированных символов
<code>lsp_restart</code>	Перезапуск конкретных или всех LSP-клиентов
<code>job_output</code>	Получение <code>stdout/stderr</code> фоновых процессов
<code>job_kill</code>	Завершение фоновых процессов
<code>todos</code>	Встроенное отслеживание задач (<code>pending/in_progress/completed</code>)
<code>list_mcp_resources</code>	Список ресурсов MCP-сервера
<code>read_mcp_resource</code>	Чтение конкретного ресурса MCP

Поддержка провайдеров: Мультипровайдерная через фреймворк `Fantasy` — `OpenAI`, `Anthropic`, `Google`, `Azure`, `Bedrock`, а также `Hyper` (собственный прокси `Charmbracelet` на `hyper.charm.land`). `Hyper` поддерживает режимы `JSON` и потоковой передачи, повторные попытки с `Retry-After`, отслеживание кредитов.

Интеграция MCP (полная):

- Три транспорта: `stdio`, `HTTP` (`StreamableClient`), `SSE`
- Четыре состояния: `Disabled` → `Starting` → `Connected` → `Error` (публикуются через `pubsub`)
- Полная поддержка: инструменты, ресурсы, промпты
- Именование инструментов: `mcp_[serverName]_[toolName]`
- Проверка разрешений перед каждым выполнением MCP-инструмента
- Валидация поддержки изображений/медиа для каждой модели

Интеграция с LSP (глубокая):

- `lsp.Manager` управляет несколькими клиентами языковых серверов
- Диагностика: сортировка по серьёзности, лимит 10 на категорию, по файлу + по всему проекту
- Ссылки: предварительная фильтрация через `grep` → LSP `FindReferences`, поддержка квалифицированных символов (`::`, `..`, `\\`)
- Перезапуск LSP с конкурентными горутинами
- Настройка в `crush.json` (`Go`, `TypeScript`, `Nix`, любой LSP-сервер)

TUI: Полный стек Charm — Bubble Tea (фреймворк), Lip Gloss (стилизация), Glamour (markdown), Bubbles (компоненты).

Управление сессиями: Coordinator управляет сессиями. Автоматическое резюмирование, генерация заголовков. Контекст разрешений на уровне сессии.

Система разрешений: Многоуровневая:

- Bash: чёрный список команд + блокировщики аргументов + белый список безопасных команд
- Файлы: обязательное чтение перед редактированием, проверка времени модификации, разрешение символических ссылок, разрешение на работу вне рабочей директории
- MCP: разрешения для каждого инструмента, фильтрация отключённых инструментов
- Глобально: флаг `--yo1o` отключает все проверки, белые списки в `crush.json`

Обнаружение зацикливания: SHA-256 хеш от (имя инструмента + вход + выход) по скользящему окну из 10 шагов. Срабатывает, если любая сигнатура повторится 5+ раз.

Система промптов: Go text/template со встраиванием на этапе компиляции. Три шаблона: `coder` (основной), `task` (подагент), `initialize`. Внедряет статус `git`, ветку, последние коммиты, навыки в формате XML, контекстные файлы.

Система навыков: Обнаруживаемые пакеты из `~/.config/crush/skills/` и пользовательских путей. Сериализуются как XML в промпты.

Уникальные возможности:

1. Провайдер Huper (LLM-прокси Charmbracelet)
2. Поиск по репозиториям через Sourcegraph
3. Агентный fetch (выделенный подагент для веб-исследований)
4. Фоновые задачи с автоматическим переводом в фон (порог 60 с)
5. Встроенное отслеживание задач
6. Поиск через DuckDuckGo (без API-ключа)
7. Инструмент multi-edit
8. Фреймворк Fantasy (charm.land/fantasy)
9. Поддержка `.crushignore`
10. Контекст проекта через [AGENTS.md](#)

2.3 OpenAI Codex CLI (openai/codex)

Обзор: Двухуровневая архитектура — устаревший CLI на TypeScript (заменён) и актуальный CLI на Rust. ~49,3K звёзд, Apache-2.0.

Архитектура:

Рабочее пространство Rust (`codex-rs/`):

- `core/` — библиотека бизнес-логики
- `exec/` — headless CLI
- `tui/` — TUI на Ratatui
- `cli/` — мультиинструментный бинарник
- `app-server-protocol/` — JSON-RPC схема

Множество поверхностей развёртывания: CLI, расширение для IDE (VS Code/Cursor/Windsurf), Codex App (десктоп macOS), Codex Web (chatgpt.com/codex). Распространяется через npm (`@openai/codex`), Homebrew, бинарники GitHub Release.

Цикл агента: Цикл «думай-действуй-наблюдай» в стиле ReAct в `AgentLoop` . Ограничения структурированного вывода предотвращают галлюцинации. Самовосстановление: `stderr` подаётся обратно как контекст. Оптимизация токенов: усечение, скользящее окно (3–5 взаимодействий), пакетирование.

Инструменты (минимальный, но мощный набор):

Инструмент	Описание
<code>apply_patch</code>	Виртуальный CLI через крейт <code>codex-arg0</code> , унифицированные diff-патчи
<code>local_shell_call</code>	Оболочка в песочнице с массивом команд, <code>cwd</code> , <code>env</code> , тайм-аутом
<code>web_search_call</code>	Веб-поиск
<code>function_call</code>	Стандартный вызов функций OpenAI

Инструмент	Описание
custom_tool_call	MCP/динамические инструменты через DynamicToolSpec

Поддержка провайдеров (10+): OpenAI (по умолчанию, o4-mini), OpenRouter, Azure, Gemini, Ollama, Mistral, DeepSeek, xAI, Groq, ArceeAI, а также любой API, совместимый с OpenAI. Использует Responses API.

MCP (двойная роль):

- Клиент: подключается к серверам из config.toml
- Сервер: codex mcp-server (экспериментально) — позволяет другим агентам использовать Codex как инструмент
- Управление: codex mcp add/list/get/remove
- Поддержка OAuth для MCP-серверов

LSP: Нет нативной интеграции с LSP.

TUI: Терминальный интерфейс на Ratatui (Rust).

Управление сессиями: БД состояния на SQLite (codex_sqlite_номе). Ghost-коммиты для невидимых снимков репозитория. Компактификация с шифрованием содержимого. Режим планирования с отдельным уровнем рассуждений. codex exec --ephemeral для запусков без сохранения.

Модель песочницы (лучшая в классе):

- macOS: Apple Seatbelt (sandbox-exec) — jail только для чтения, настраиваемые корни для записи, сеть полностью заблокирована
- Linux: Landlock LSM через крейт codex-arg0 с самоперезапуском; Docker рекомендуется для более строгой изоляции
- Windows: только WSL2
- Три режима: только чтение (по умолчанию), запись в рабочую директорию, полный доступ (опасный)
- Full Auto = сеть отключена + ограничение директорией

Система разрешений: Перечисление AskForApproval : untrusted, on-failure, on-request, never, reject.

Протокол App-Server: JSON-RPC между фронтендами и ядром бэкенда. Ключевые типы: InitializeParams, CommandExecParams, ConfigRead/Write, FuzzyFileSearch, ReviewStartParams. Перечисления: SandboxMode, AskForApproval, ReasoningEffort (none→xhigh), Personality (none/friendly/pragmatic).

Конфигурация: TOML в ~/.codex/. Многоуровневая: пользовательская → проектная. Иерархия AGENTS.md: ~/.codex/AGENTS.md → корень репозитория → текущая директория. Обнаружение и импорт конфигурации внешних агентов.

Система навыков: На основе файловой системы через .codex/skills/<name>/. SKILL.md с YAML-метаданными. Встроенные: babysit-pr (автономный мониторинг PR), test-tui. Codex App расширяет навыки до автоматизаций (запланированные фоновые задачи).

Уникальные возможности:

1. Ghost-коммиты (невидимые снимки репозитория)
2. Навык babysit-pr (автономный DevOps-агент с классификацией сбоев CI)
3. Codex как MCP-сервер (мета-агентные архитектуры)
4. Миграция конфигурации внешних агентов
5. Система личности (none/friendly/pragmatic)
6. Встроенный код-ревью (uncommittedChanges/baseBranch/commit/пользовательские цели)
7. Режим планирования с настраиваемым уровнем рассуждений
8. Нативная производительность Rust
9. Лучшая в классе песочница (Seatbelt + Landlock)
10. Множество поверхностей развёртывания (CLI, IDE, App, Web)

2.4 Claude Code (Anthropic)

Обзор: Официальный CLI Anthropic для Claude. TypeScript, ~40,8K звёзд. Эталонная реализация AI-агентов для кодирования.

Архитектура: CLI на Node.js с системой хуков, интеграцией MCP, навыками и порождением подагентов через инструмент Task.

Инструменты (10+): Read, Edit, Write, Glob, Grep, Bash, Task (подагенты), WebFetch, WebSearch, NotebookEdit, AskUserQuestion, EnterPlanMode, а также MCP-инструменты.

Поддержка провайдеров: Только модели Anthropic (Claude Opus 4.6, Sonnet 4.6, Haiku 4.5). Маршрутизация моделей: Opus для архитектуры, Sonnet для функциональности, Haiku для быстрых задач.

MCP: Полная клиентская поддержка. Инструменты, ресурсы, промпты. Настройка в settings.json.

LSP: Нет нативного LSP (полагается на интеграцию IDE при использовании в качестве расширения).

TUI: Терминальный чат-интерфейс с рендерингом markdown.

Управление сессиями: Файлы транскриптов в формате JSONL. Компактификация контекста при ~120K токенов. Хуки pre-compact для обеспечения непрерывности.

Система разрешений: Режимы разрешений с подтверждением на уровне инструментов. Хуки могут перехватывать и разрешать/запрещать/запрашивать. [CLAUDE.md](#) для инструкций проекта.

Система хуков (14 событий): SessionStart, UserPromptSubmit, PreToolUse, PermissionRequest, PostToolUse, PostToolUseFailure, Notification, SubagentStart, SubagentStop, Stop, TeammateIdle, TaskCompleted, PreCompact, SessionEnd. Хуки могут внедрять контекст, блокировать действия, изменять разрешения.

Уникальные возможности:

1. Наиболее зрелая система хуков/жизненного цикла (14 событий)
2. Порождение подагентов с изолированным контекстом (инструмент Task)
3. Система навыков (вызываемые пользователем слеш-команды)
4. Инструкции проекта через [CLAUDE.md](#) (иерархические)
5. Компактификация контекста с хуками pre-compact
6. Режимы разрешений с гранулярным контролем на уровне инструментов
7. Поддержка редактирования ноутбуков
8. Режим планирования с потоком одобрения пользователем

2.5 Gemini CLI (Google)

Обзор: Терминальный AI-агент от Google. TypeScript, ~95,7K звёзд (наибольшее количество в категории). Apache-2.0.

Ключевые характеристики:

- Контекстное окно в 1M токенов (наибольшее с большим отрывом)
- Gemini 2.5 Pro / Gemini Flash (только модели Google)
- Поддержка MCP (локальные и удалённые серверы)
- Контрольные точки диалогов (сохранение/возобновление)
- Система безопасности Trusted Folders
- Конфигурация проекта через [GEMINI.md](#)
- Мультимодальный ввод (эскизы, PDF)
- Заземление через Google Search
- Бесплатный тариф: 60 запросов/мин, 1K запросов/день

Уникальные возможности: Огромное контекстное окно, щедрый бесплатный тариф, интеграция с Google Search, мультимодальный ввод.

2.6 Aider

Обзор: CLI для парного программирования с AI. Python, ~40,9K звёзд. Apache-2.0. Пионер концепции карты репозитория (repo-map).

Ключевые характеристики:

- Карта репозитория на основе tree-sitter для понимания кодовой базы
- Автоматические git-коммиты как контрольные точки
- Голосовое кодирование (voice-to-code)
- Режим наблюдения IDE
- Линтинг/тестирование с автоисправлением
- Поддержка 100+ языков
- Форматы редактирования: whole/diff/udiff
- Режим архитектора (рабочий процесс с двумя моделями)
- Лидер SWE-bench
- Нет нативного MCP (существует форк сообщества mcpm-aider)
- Нет нативного LSP (использует tree-sitter)
- Поддержка моделей: Claude, DeepSeek, GPT-4o, o1, o3-mini, практически любая LLM через Ollama

Уникальные возможности: Пионер карты репозитория, git как управление сессиями, режим архитектора, голосовое кодирование, самая широкая языковая поддержка.

2.7 Goose (Block/Square)

Обзор: Расширяемый AI-агент. Rust (59%) + TypeScript (33%), ~31,2K звёзд. Apache-2.0.

Ключевые характеристики:

- Расширения ЯВЛЯЮТСЯ MCP-серверами (MCP-архитектура первого класса)
- CLI + десктопное GUI-приложение
- Любой провайдер LLM, несколько конфигураций одновременно
- Система рецептов для komponуемых рабочих процессов
- Режим наставника для обучения
- 113 релизов (v1.25.0)
- История диалогов на основе сессий

Уникальные возможности: Архитектура «расширения как MCP-серверы», двойной интерфейс CLI+десктоп, рецепты, режим наставника.

2.8 Дополнительные заметные инструменты

Cline (~58,4K звёзд) — Расширение для VS Code. Автономное многошаговое выполнение, автоматизация браузера через Playwright, возможность Computer Use, контрольные точки рабочего пространства, создание MCP-инструментов по запросу. Форки: Roo Code, Kilo Code.

Continue.dev (~31,5K звёзд) — Расширение для VS Code/JetBrains + CLI. AI-проверки как статусы GitHub, проверки как код в `.continue/checks/`, интеграция с MCP Registry.

Warp (~26K звёзд) — Терминал с GPU-ускорением. Мультиагентная оркестрация (агент Oz), поддержка CLI-агентов (Claude Code, Codex, Gemini CLI). \$200/месяц для корпоративного тарифа.

Zed (~75,9K звёзд) — Редактор на Rust с панелью агента, моделью автодополнения Zeta, ACP (Agent Client Protocol) для сторонних агентов, совместное редактирование. Полная поддержка LSP как IDE.

Amp/Sourcegraph — Закрытый исходный код. Автоматическая маршрутизация между моделями, режим Deep, подагенты Oracle/Librarian, основа на Sourcegraph code intelligence, общие потоки.

Codebuff (~2,9K звёзд) — Мультиагентная архитектура (File Picker → Planner → Editor → Reviewer), память через [knowledge.md](#), SDK для встраивания. Заявляет 61% против 53% у Claude Code на бенчмарках.

Plandex (~14,6K звёзд) — Go, огромный контекст (20M+ токенов), выполнение в песочнице.

Qwen Code (~14,9K звёзд) — Форк Gemini CLI, оптимизированный для Qwen3-Coder.

3. Матрица сравнения функциональности

Основные характеристики

Характеристика	OpenCode	Crush	Codex	Claude Code	Gemini CLI	Aider	Goose
Язык	Go	Go	Rust+TS	TypeScript	TypeScript	Python	Rust+TS
Звёзды	~30K	~14K	~49K	~41K	~96K	~41K	~31K
Лицензия	Apache-2.0	Apache-2.0	Apache-2.0	Проприетарная	Apache-2.0	Apache-2.0	Apache-2.0
TUI-фреймворк	Bubble Tea	Bubble Tea	Ratatui	Собственный	Собственный	Терминал	CLI+Десктоп

Цикл агента и инструменты

Характеристика	OpenCode	Crush	Codex	Claude Code	Gemini CLI	Aider	Goose
Цикл агента	ReAct однопоточный	ReAct однопоточный	ReAct однопоточный	ReAct однопоточный	ReAct однопоточный	ReAct однопоточный	ReAct однопоточный
Встроенные инструменты	12+	20+	2 основных + 3	10+	~8	~6	На основе MCP
Подагенты	Да (только чтение)	Да (агентный fetch)	Нет	Да (инструмент Task)	Нет	Нет (режим архитектора)	Нет
Фоновые задачи	Нет	Да (авто-фон 60 с)	Нет	Нет	Нет	Нет	Нет
Обнаружение зацикливания	Нет	Да (окно SHA-256)	Нет	Да (хуки)	Нет	Нет	Нет
Отслеживание задач	Нет	Да (встроенное)	Нет	Да (TaskCreate)	Нет	Нет	Нет

Поддержка провайдеров и моделей

Характеристика	OpenCode	Crush	Codex	Claude Code	Gemini CLI	Aider	Goose
Провайдеры	7+	Мульти + Hyper	10+	Только Anthropic	Только Google	Любая LLM	Любая LLM
Локальные модели	Через OpenAI-совместимые	Через провайдеров	Ollama	Нет	Нет	Ollama	Ollama
Маршрутизация моделей	Нет	Нет	Нет	Да (Opus/Sonnet/Haiku)	Нет	Режим архитектора	Несколько конфигураций
Собственный прокси	Нет	Hyper (charm.land)	Нет	Нет	Нет	Нет	Нет

MCP и LSP

Характеристика	OpenCode	Crush	Codex	Claude Code	Gemini CLI	Aider	Goose
MCP-клиент	Да	Да (полный)	Да	Да	Да	Нет	Да (первый класс)
MCP-сервер	Нет	Нет	Да (экспериментально)	Нет	Нет	Нет	Нет
Транспорты MCP	stdio	stdio/HTTP/SSE	stdio	stdio	stdio	—	stdio
Ресурсы MCP	Нет	Да	Нет	Да	Нет	—	Да
Промпты MCP	Нет	Да	Нет	Да	Нет	—	Нет
Диагностика LSP	Да	Да (глубокая)	Нет	Нет	Нет	Нет	Нет
Ссылки LSP	Нет	Да	Нет	Нет	Нет	Нет	Нет
Перезапуск LSP	Нет	Да	Нет	Нет	Нет	Нет	Нет

Сессии и конфигурация

Характеристика	OpenCode	Crush	Codex	Claude Code	Gemini CLI	Aider
Хранение	SQLite	Хранилище сессий	SQLite	Файлы JSONL	Контрольные точки	Git-коммиты
Формат конфигурации	JSON	JSON (crush.json)	TOML	JSON	GEMINI.md	Флаги CLI/YAML

Характеристика	OpenCode	Crush	Codex	Claude Code	Gemini CLI	Aider
Уровни конфигурации	Пользователь + Проект	Пользователь + Проект	Пользователь + Проект	Глобальный + Проект	Проект	CLI + конфиг
Файл проекта	.opencode.json	AGENTS.md	AGENTS.md	CLAUDE.md	GEMINI.md	.aider*
Контекстное окно	Стандартное	Авторезюмирование	Скользящее окно	120К + компактификация	1М токенов	Карта репозитория
Авторезюмирование	Да	Да	Да (компактификация)	Да (компактификация)	Нет (огромный контекст)	Нет

Безопасность и разрешения

Характеристика	OpenCode	Crush	Codex	Claude Code	Gemini CLI	Aider	Goose
Система разрешений	Да	Да (многоуровневая)	Да (песочница)	Да (режимы+хуки)	Trusted Folders	Нет	Governance
Песочница	Нет	Нет	Да (Seatbelt/Landlock)	Нет	Да	Нет	Нет
Чёрный список команд	Да (curl,wget,nc)	Да (обширный)	Н/П (песочница)	Нет (на основе хуков)	Нет	Нет	Нет
Чтение перед редактированием	Да	Да	Н/П	Да	Нет	Нет	Нет
Флаг пропуска проверок	Нет	--yolo	danger-full-access	Нет	Нет	Нет	Нет
Проверка времени модификации	Да	Да	Нет	Нет	Нет	Нет	Нет

Расширяемость

Характеристика	OpenCode	Crush	Codex	Claude Code	Gemini CLI	Aider	Goose
Навыки/плагины	Нет	Да (каталог навыков)	Да (SKILL.md)	Да (слеш-команды)	Нет	Нет	Рецепты
Хуки/жизненный цикл	Нет	Нет	Нет	Да (14 событий)	Нет	Нет	Нет
Веб-поиск	Нет	DuckDuckGo	Да	Инструмент WebSearch	Google Search	Нет	Через MCF
Sourcegraph	Да	Да	Нет	Нет	Нет	Нет	Нет
Интеграция с Git	Базовая	Да (атрибуция)	Ghost-коммиты	Через хуки	Базовая	Глубокая (автокоммиты)	Базовая
Код-ревью	Нет	Нет	Да (встроенный)	Нет	Нет	Нет	Нет
Система diff	Унифицированный diff	Edit + MultiEdit	apply_patch (унифицированный)	Edit (замена строк)	Неизвестно	whole/diff/udiff	Неизвестн

4. Анализ лучших в своём классе

Категория	Лучший инструмент	Почему
Песочница	Codex	Единственный инструмент с песочницей на уровне ОС (Seatbelt на macOS, Landlock на Linux). Сеть полностью заблокирована в автоматическом режиме. Остальные полагаются на запросы разрешений.
Интеграция с LSP	Crush	Наиболее глубокая интеграция с LSP: диагностика, ссылки, перезапуск. Менеджер нескольких клиентов. Инструмент view автоматически добавляет диагностику. OpenCode имеет только диагностику.
Поддержка MCP	Crush / Goose	Crush: полная спецификация (инструменты+ресурсы+промпты), 3 транспорта, конечный автомат. Goose: расширения ЯВЛЯЮТСЯ MCP-серверами. Codex уникален как MCP-сервер.
Широта инструментов	Crush	20+ инструментов, включая фоновые задачи, отслеживание задач, multi-edit, агентный fetch, DuckDuckGo, Sourcegraph. Наиболее полный встроенный набор инструментов.
Мультипровайдерность	Codex	10+ встроенных провайдеров плюс любой API, совместимый с OpenAI. Самая широкая поддержка из коробки.
Управление контекстом	Gemini CLI	Окно в 1M токенов устраняет необходимость в компактификации/резюмировании. Компактификация Claude Code на основе хуков — наиболее изощённая для меньших окон.
Хуки/жизненный цикл	Claude Code	14 событий жизненного цикла с полным перехватом. Ни один другой инструмент не приближается. Crush и OpenCode не имеют поддержки хуков.
Подагенты	Claude Code	Инструмент Task с изолированным контекстом, выбором модели (Opus/Sonnet/Haiku), параллельным выполнением. Crush имеет агентный fetch. OpenCode имеет подагент только для чтения.
Качество TUI	Crush / OpenCode	Оба используют Bubble Tea (лучший TUI-фреймворк для Go). Crush добавляет Lip Gloss + Glamour для стилизации/markdown. Codex использует Ratatui (аналог для Rust).
Интеграция с Git	Aider	Автокоммиты как контрольные точки, git как управление сессиями. Codex имеет ghost-коммиты. Остальные — базовая интеграция.
Понимание репозитория	Aider	Пионер карт репозитория на основе tree-sitter. Выбор контекста на основе AST для 100+ языков. Остальные используют grep/glob.
Модель безопасности	Codex	Песочница на уровне ОС + режимы одобрения + блокировка сети. Crush имеет лучшую модель на основе разрешений (многоуровневая, проверка времени модификации).
Навыки/расширяемость	Claude Code	Навыки + хуки + MCP = наибольшая расширяемость. Codex имеет SKILL.md + автоматизации. Crush имеет каталог навыков.
Фоновое выполнение	Crush	Единственный инструмент с автоматическим переводом в фон (порог 60 с), отслеживанием задач, завершением задач. Необходимо для dev-серверов и длительных сборок.
Код-ревью	Codex	Встроенный ревью с несколькими целями (незакоммиченные изменения, ветка, коммит, пользовательские). Ни один другой CLI не имеет нативного код-ревью.
Поиск по репозиториям	OpenCode / Crush	Оба интегрируют Sourcegraph. Crush также имеет DuckDuckGo. Остальные не имеют возможности поиска по нескольким репозиториям.
Обнаружение зацикливания	Crush	Хеширование сигнатур SHA-256 по скользящему окну. Просто и эффективно. Claude Code реализует это через хуки (более гибко, но внешне).
Инженерия промптов	Crush	Система шаблонов Go с встраиванием на этапе компиляции, внедрением контекста git, сериализацией навыков в XML. Наиболее структурированный подход.
Производительность	Codex	Нативный бинарник на Rust. Самый быстрый запуск и выполнение. Инструменты на Go (OpenCode, Crush) — второй эшелон. TS/Python — самые медленные.
Бесплатный тариф	Gemini CLI	60 запросов/мин, 1K запросов/день бесплатно. Ни один другой инструмент не предлагает сопоставимого бесплатного доступа.

5. Рекомендации для ss-cli

На основе данного анализа следующие функции представляют наибольшую ценность для реализации нового CLI-агента:

Обязательный минимум (базовые требования)

1. **Абстракция мультипровайдерности** — Поддержка минимум 5 провайдеров (OpenAI, Anthropic, Google, локальные через Ollama, OpenRouter). Использовать паттерн интерфейса провайдера по образцу OpenCode/Crush.
2. **Клиентская поддержка MCP** — Полная спецификация: инструменты + ресурсы + промпты. Поддержка транспорта stdio как минимум, HTTP/SSE для удалённых серверов. Следовать паттерну конечного автомата Crush (Disabled→Starting→Connected→Error).
3. **Система разрешений** — Многоуровневая по образцу Crush: чёрный список команд, белый список безопасных команд, обязательное чтение перед редактированием, проверка времени модификации. Добавить аналог `--yo1o` для опытных пользователей.
4. **Сохранение сессий** — На основе SQLite (как OpenCode/Codex). Хранение сообщений, вызовов инструментов, метаданных. Поддержка возобновления сессий.
5. **Файл конфигурации проекта** — [AGENTS.md](#) или аналог с иерархической загрузкой (глобальный → репозиторий → текущая директория).

Высокоценные отличительные возможности

6. **Интеграция с LSP** — Следовать модели Crush: диагностика + ссылки + перезапуск. Менеджер нескольких клиентов. Автоматическое добавление диагностики при просмотре файлов. Запуск диагностики после записи/патчей. Это существенное конкурентное преимущество — только 2 из 15 инструментов имеют его.
7. **Система хуков/жизненного цикла** — Система из 14 событий Claude Code является золотым стандартом. Начать с 5–6 основных событий: SessionStart, PreToolUse, PostToolUse, PreCompact, Stop. Обеспечить внедрение контекста и перехват разрешений.
8. **Управление фоновыми задачами** — Паттерн автоматического перевода в фон от Crush (порог 60 с) элегантен. Необходим для dev-серверов, сборок, тестовых наборов. Включить `job_output` и `job_kill`.
9. **Обнаружение заикливания** — Скользящее окно SHA-256 от Crush — простое и эффективное решение. Окно из 10 шагов, порог повторения 5 раз.
10. **Порождение подагентов** — Изолированные контекстные окна для параллельных исследований. Подмножество инструментов только для чтения для безопасности. Инструмент Task от Claude Code является эталоном.

Желательные возможности (конкурентное преимущество)

11. **Интеграция с Sourcegraph** — Поиск по нескольким репозиториям эффективен для понимания паттернов использования библиотек. И OpenCode, и Crush имеют его. GraphQL API, несложная реализация.
12. **Паттерн «Codex как MCP-сервер»** — Предоставление самого агента в качестве MCP-сервера обеспечивает мета-агентные архитектуры. Уникально для Codex, высокая инновационная ценность.
13. **Песочница на уровне ОС** — Seatbelt (macOS) + Landlock (Linux) по образцу Codex. Значительно надёжнее запросов разрешений. Сложно в реализации, но лучшее в классе решение для безопасности.
14. **Встроенный код-ревью** — Ревью Codex с несколькими целями (незакоммиченные изменения, ветка, коммит). Естественная интеграция в рабочий процесс.
15. **Система навыков/плагинов** — На основе файловой системы по образцу Codex ([SKILL.md](#)) или Crush (каталог навыков). Обеспечить расширяемость сообществом.
16. **Отслеживание задач** — Встроенная система задач Crush даёт пользователям видимость многошаговых операций. Простая реализация, высокая ценность для UX.
17. **Агентный fetch** — Выделенный подагент Crush для веб-исследований со своим набором инструментов. Лучше, чем простой fetch для исследовательских задач.
18. **Ghost-коммиты** — Невидимые снимки репозитория Codex для отката. Элегантный механизм отмены.
19. **Авторезюмирование** — И OpenCode, и Crush автоматически резюмируют сессии. Необходимо для управления контекстом при меньших окнах.
20. **Система шаблонов промптов** — Подход Crush с шаблонами Go и встраиванием на этапе компиляции, внедрением контекста git и сериализацией навыков. Структурированный и поддерживаемый.

Рекомендации по архитектуре

- **Язык:** Go или Rust. Go имеет лучшую экосистему для TUI (Bubble Tea) и более быструю разработку. Rust — для максимальной производительности и песочницы.
- **TUI:** Bubble Tea (Go) или Ratatui (Rust). Оба зрелые, хорошо документированные.
- **БД:** SQLite для сессий, конфигурации и состояния. Проверено OpenCode, Codex и нашими memory-tools.
- **Конфигурация:** TOML (как Codex) или JSON (как OpenCode). Многоуровневая: пользовательская → проектная.

- **Цикл агента:** Однопоточный ReAct с продолжением по использованию инструментов. Последовательное выполнение инструментов с проверкой разрешений.
- **Интерфейс провайдера:** Абстрактный провайдер с методами Generate() и Stream(). Паттерн функциональных опций (как Hyper в Crush).

6. Ссылки на базу знаний

Идентификаторы источников

Источник	ID	Элементов	Описание
OpenCode	262	10	Архитектура, инструменты, цикл агента, провайдеры
Crush	263	14	Архитектура, инструменты, MCP, LSP, разрешения, промпты, уникальные возможности
awesome-opencode	264	10	Обзор ландшафта: Gemini CLI, Aider, Goose, Cline, Continue, Warp, Zed, Amp, Codebuff
Codex	265	8	Архитектура, инструменты, песочница, MCP, конфигурация, навыки, протокол, уникальные возможности

Ключевые идентификаторы элементов базы знаний по темам

OpenCode:

- 8368 — Обзор архитектуры
- 8369 — Цикл агента
- 8370 — Система инструментов
- 8371 — Инструмент Bash
- 8373 — Инструмент Patch
- 8375 — Инструмент Write
- 8376 — Инструмент Grep

Crush:

- 8396 — Обзор архитектуры
- 8399 — Обнаружение зацикливания
- 8400 — Провайдер Hyper
- 8401 — Интеграция MCP
- 8402 — Система промптов
- 8403 — Фоновые задачи
- 8404 — Интеграция LSP
- 8405 — Sourcegraph
- 8406 — Система задач
- 8407 — Безопасность и разрешения
- 8409 — Сравнение уникальных возможностей

Codex:

- 8388 — Обзор архитектуры
- 8389 — Протокол App-Server
- 8390 — Модель песочницы
- 8391 — Система навыков
- 8392 — Инструменты и цикл агента
- 8393 — Мультимодельность и MCP
- 8394 — Конфигурация и сессии
- 8395 — Уникальные возможности, сильные и слабые стороны

Ландшафт (awesome-opencode):

- 8378 — Aider
- 8379 — Goose
- 8380 — Gemini CLI
- 8381 — Cline

- 8382 — Continue.dev
- 8383 — Amp/Sourcegraph
- 8384 — Codebuff
- 8385 — Zed
- 8386 — Warp

Поисковые запросы для извлечения из базы знаний

```
mem_kb({action:"search", query:"OpenCode tools architecture"})  
mem_kb({action:"search", query:"Crush MCP LSP permissions"})  
mem_kb({action:"search", query:"Codex sandbox skills protocol"})  
mem_kb({action:"search", query:"AI agent CLI comparison landscape"})
```

Отчёт сформирован на основе 42 элементов базы знаний из 4 просканированных репозиториях и веб-исследования. Все данные сохранены в векторной БД memory-tools для дальнейшего использования.